ActionDispatcher 动作分发模块单元测试报告

测试对象：`dispatcher.py` (ActionDispatcher 类)

测试框架：`pytest` + `unittest.mock`

一、 测试目标与策略

本次测试旨在全面验证动作分发模块（ActionDispatcher）在处理各类 UI 自动化动作指令时的正确性、鲁棒性及容错能力。
由于该模块底层依赖 `pyautogui` 和 `mss` 等系统级库，直接运行会产生真实的鼠标移动、点击和键盘输入，干扰正常测试。因此，本次测试采用 **Mock 技术** 隔离了所有底层系统调用，实现了无物理副作用的纯逻辑验证。同时，针对底层调用方法（如 `moveTo`，`click`，`keyDown` 等）的执行情况及参数传递进行了精准断言。

二、 测试用例维度设计

针对不同的 Action 类型，测试用例严格按照四个维度进行覆盖设计，确保边界情况与异常输入均被妥善处理：

1. **正常用例**
   - 验证标准输入下的预期行为。例如：带坐标/不带坐标的点击、带修饰键的组合操作、组合键的按下与释放、混合大小写文本输入等。
2. **边界用例**
   - 验证零坐标 `(0,0)`、屏幕最大分辨率边界、以及严重越界坐标下的处理逻辑，确保不引发崩溃。
   - 验证拖拽起点和终点重合（原地拖拽）、连续两次按下同一按键、连续两次松开鼠标、修饰键传入空数组 `[]` 等边缘状态。
   - 验证传入大写键名（如 `"ENTER"`，`"Ctrl"`）时，代码能否正确归一化为小写并执行。
3. **非法用例**
   - 输入错误的参数类型，如坐标传入字符串 `"100"` 或布尔值、修饰键传入整数数组、按键传入 `None` 或整数，代码应抛出异常。
   - 输入不支持的按键名称（如 `"nokey"`）、非法的滚动方向（如 `"diagonal"`）等，代码应抛出异常。
4. **缺失用例**
   - **必填项校验**：验证缺少核心参数时的行为。例如：只传了 `x` 未传 `y`、拖拽操作缺失终点坐标 `end_x/end_y`、`type_text` 缺失 `text` 字段等，代码应抛出异常。

三、 测试代码工程化亮点

1. **数据驱动测试 (Data-Driven Testing)**：
   大量使用了 `@pytest.mark.parametrize` 装饰器。例如，左键、右键、中键的点击逻辑高度相似，通过参数化批量传入不同的 `atype` 和 `button`，避免了测试代码的冗余和复制粘贴，提升了代码的可维护性。
2. **精确的 Mock 行为断言**：
   测试不仅断言了最终的返回值 `{"type": "ok"}`，还深入验证了底层 API 的调用细节。例如：
   - 传入坐标时，断言 `moveTo(x, y)` 被准确调用。

- 不传坐标时，断言 `moveTo` 绝对没有被调用。
- 验证快捷键（如 `ctrl+c`）执行时，`keyDown` 和 `keyUp` 的调用顺序与次数。

3. **清晰的测试类划分**：
   代码根据 Action 的类别划分了 `TestScreenshot`，`TestClicks`，`TestScroll`，`TestKeyPress` 等测试类，每个类使用 `setup_method` 进行前置初始化，结构清晰，便于后续维护与扩展。

四、 测试结果总结

本次测试覆盖了截屏、获取光标位置、各类鼠标点击、滚轮滑动、左键拖拽、鼠标按下/抬起、键盘按键/按下/抬起、文本输入等所有核心动作。

- **用例执行情况**：全部测试用例 100% 通过。

附录：测试用例矩阵明细

为了确保测试覆盖的全面性，避免遗漏重要的分支逻辑，我针对 ActionDispatcher 支持的 17 种基础操作，逐一进行了测试用例的精细化拆解。下表展示了各个动作在正常、边界、非法和缺失四个维度下的具体测试场景设计：

操作类型 (Action Type)	正常用例 (Normal)	边界用例 (Edge)	非法用例 (Invalid)	缺失用例 (Missing)
screenshot 截屏	1. `{"type":"screenshot"}` → `{"type":"image", "data":<png>}`	不适用	不适用	不适用
cursor_position 获取光标位置	1. Mock `position()` → `(100,200)` → `{"type":"text","data":"[100, 200]"}`	不适用	不适用	不适用
mouse_move 移动鼠标	1. `{"type":"mouse_move","x":100,"y":200}` → `{"type":"ok"}`	1. 坐标全为 0 2. 最大有效坐标 3. 越界坐标	1. 坐标为字符串 (String)	1. 缺少 `x` 和 `y` 2. 仅缺 `x` 3. 仅缺 `y`

left_click 左键单击	1. 无坐标无修饰键 2. 有坐标无修饰键 3. 有坐标有修饰键 4. 仅含 `x` 缺 `y` 5. 仅含 `y` 缺 `x`	1. 坐标全为 0 2. 最大有效坐标 3. 越界坐标 4. 修饰键为空数组 `[]` 5. 修饰键名大小写	1. 坐标为字符串 2. 坐标为空 (Null) 3. 坐标为布尔值 (Boolean) 4. 修饰键为字符串 5. 修饰键为整数数组 6. 修饰键无法识别	不适用
right_click 右键单击	1. 无坐标无修饰键 2. 有坐标无修饰键 3. 有坐标有修饰键 4. 仅含 `x` 缺 `y` 5. 仅含 `y` 缺 `x`	1. 坐标全为 0 2. 最大有效坐标 3. 越界坐标 4. 修饰键为空数组 `[]` 5. 修饰键名大小写	1. 坐标为字符串 2. 坐标为空 (Null) 3. 坐标为布尔值 4. 修饰键为字符串 5. 修饰键为整数数组 6. 修饰键无法识别	不适用
middle_click 中键单击	1. 无坐标无修饰键 2. 有坐标无修饰键 3. 有坐标有修饰键 4. 仅含 `x` 缺 `y` 5. 仅含 `y` 缺 `x`	1. 坐标全为 0 2. 最大有效坐标 3. 越界坐标 4. 修饰键为空数组 `[]` 5. 修饰键名大小写	1. 坐标为字符串 2. 坐标为空 (Null) 3. 坐标为布尔值 4. 修饰键为字符串 5. 修饰键为整数数组 6. 修饰键无法识别	不适用

left_double_click 左键双击	1. 无坐标无修饰键 2. 有坐标无修饰键 3. 有坐标有修饰键 4. 仅含 `x` 缺 `y` 5. 仅含 `y` 缺 `x`	1. 坐标全为 0 2. 最大有效坐标 3. 越界坐标 4. 修饰键为空数组 `[]` 5. 修饰键名大小写	1. 坐标为字符串 2. 坐标为空 (Null) 3. 坐标为布尔值 4. 修饰键为字符串 5. 修饰键为整数数组 6. 修饰键无法识别	不适用
scroll 滚动滚轮	1. 仅包含方向 (上/下/左/右) 2. 仅包含方向和滑动幅度 3. 包含方向、幅度及坐标 4. 有方向且仅含 `x` 缺 `y` 5. 有方向且仅含 `y` 缺 `x`	1. 滑动幅度为 0 2. 滑动幅度极大 3. 滑动幅度越界	1. 滑动幅度非法 2. 滑动方向非法	1. 未填方向
left_click_drag 左键拖拽	1. 仅含终点 2. 包含起点和终点 3. 包含起点、终点及修饰键 4. 终点仅含 `x` 缺 `y` 5. 终点仅含 `y` 缺 `x`	1. 起点和终点重合 (原地拖拽) 2. 起点或终点坐标为 0 3. 起点或终点越界 4. 修饰键为空数组 `[]` 5. 修饰键名大小写	1. 坐标为字符串 2. 坐标为空 (Null) 3. 坐标为布尔值 4. 修饰键为字符串 5. 修饰键为整数数组 6. 修饰键无法识别	1. 缺终点 `x` 2. 缺终点 `y` 3. 缺终点 `x` 和 `y`

left_mouse_down 左键按下	1. `{"type":"left_mouse_down"}` → `{"type":"ok"}`	1. 连续按下两次 2. 按下后不松开再次按下	不适用	不适用
left_mouse_up 左键松开	1. `{"type":"left_mouse_up"}` → `{"type":"ok"}`	1. 连续松开两次 2. 未按下直接松开	不适用	不适用
right_mouse_down 右键按下	1. `{"type":"right_mouse_down"}` → `{"type":"ok"}`	1. 连续按下两次 2. 按下后不松开再次按下	不适用	不适用
right_mouse_up 右键松开	1. `{"type":"right_mouse_up"}` → `{"type":"ok"}`	1. 连续松开两次 2. 未按下直接松开	不适用	不适用
middle_mouse_down 中键按下	1. `{"type":"middle_mouse_down"}` → `{"type":"ok"}`	1. 连续按下两次 2. 按下后不松开再次按下	不适用	不适用
middle_mouse_up 中键松开	1. `{"type":"middle_mouse_up"}` → `{"type":"ok"}`	1. 连续松开两次 2. 未按下直接松开	不适用	不适用

key_press 按键按下	1. 单个按键 2. 多个按键组合 3. 功能键 (如 "enter", "f5")	1. 重复按键 2. 键名大小写	1. `key` 为 `None` 2. `key` 为整数 3. `key` 为字符串 4. `key` 为空数组	1. 无法访问 `key` 键
key_down 键盘按下	1. 单个按键 2. 功能键 (如 "enter")	1. 连续按下两次，中间不松开 2. 键名大小写	1. `key` 为 `None` 2. `key` 为整数 3. `key` 为数组	1. 无法访问 `key` 键
key_up 键盘松开	1. 单个按键 2. 功能键 (如 "enter")	1. 连续松开两次，中间不按下 2. 键名大小写	1. `key` 为 `None` 2. `key` 为整数 3. `key` 为数组	1. 无法访问 `key` 键
type_text 输入文本	1. 输入一段文本 2. 大小写混合文本	1. 中文和表情符号 2. 带换行符和制表符	1. `text` 为 `None` 2. `text` 为整数 3. `text` 为数组 4. `text` 为空字符串	1. 无法访问 `text` 键